Automatic Differentiation

Jakub Vokoun

***Abstract*—This paper provides an explanation of Automatic Differentiation (AD) concept for university students at near-bachelor level. Includes comparison with other differentiation methods, very simple AD implementation example and more realistic usage in python library.**

***Index Terms*—autodiff, python, computational mathematics**

I. Introduction

Derivative is an indispensable tool in almost any field of mathematics. Its useful property lies in revealing the rate of change of any function. The process of differentiation is very simple, therefore presumably easily encoded into computer program. Since the birth of machine computing various methods were discovered, all with their respective drawbacks. We will explore these methods later. It is necessary to understand the foundational challenges and inner workings of differentiation first.

Derivative may be defined as a limit, such as

$$L = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h} \tag{1}$$

where $f(x)$ is differentiable on an open interval containing point a, and if the limit L exists. In principle, the derivative of any function (if exists) may be computed using this definition. However, since was discovered different approach. When a few simple functions have known derivatives, using them with a set of rules for obtaining derivatives of more complicated functions results in a different, simpler method. The process of finding a derivative is called differentiation.

For example, if we know the derivative of a polynomial

$$(x^a)' = ax^{a-1} \tag{2}$$

where a is an arbitrary integer, and a sum rule

$$(\alpha f + \beta g)' = \alpha f' + \beta g' \tag{3}$$

for any function f, g, and all real numbers α, β, we may obtain derivative for this function:

$$f(x) = 3x^2 + 5x \tag{4}$$

Using both the rules its apparent the result will be:

$$f(x)' = 6x + 5 \tag{5}$$

This simple example demonstrated, that with as little as one known derivative and one rule, we are able to differentiate a whole class of functions, polynomials.

If we add to our set of known derivatives and rules, we are quickly able to differentiate any known function. With large enough set it is only matter of mechanicaly applying rules and known derivatives in the correct order. Something a computer is supposedly very good at, or at least less error-prone than most humans.

The most powerful property of computer is the `if-else` branching logic. Another important part is the ability to perform sequential instructions. This trait might be easily exploitable in order to create a computer program which would take any function as an input and produce its derivative as output.

Most often the function we need to differentiate is in the form

$$f : \mathbb{R}^n \to \mathbb{R}^m \tag{6}$$

where n and m are any positive integers. It might be useful to calculate its partial derivatives or the Jacobian matrix. For some tasks it might be sufficient to only calculate only one partial derivative, a row or column of derivatives.

II. Alternative differentiation methods

Automatic differentiation was not the first method for finding derivatives utilizing computer. In fact it is conceptually more complex than the previous methods.

That is not to say the *older* methods are inherently worse because they were discovered sooner or that are less complex. Every method is useful in particular scenario, however it is important to know about all of them and their advantages and weaknesses.

A. Finite difference method

This method is based on the Eq. (1). The assumption is, that with a small-enough h, the error will be negligible. It is not an analytical, a precise method, rather a numerical tool for obtaining *good-enough* derivative. This method doesn't take advantage of computers properties. It only uses the machine as a mere calculator.

Humans represent numbers most often using arabic numerals. Computers represent and store numbers in different formats. When working with non-integers a floating point arithmetic was introduced, and later standardized [1]. This format has limited precision and for very large or very small (near to zero) numbers is prone to errors.

The great issue arises as the requirement on precision increases. For simplicity we will define our own data type, which has the same drawbacks as `float` but is easier to understand for humans. We will call it `FD2` as *floating decimal 2*, because it is normal arithmetic with precision up to two numbers.

If we tried to calculate derivative for $f(x) = x$, $a = 100$ and $h = 0.01$

$$f(x)' = \frac{f(100 + 0.01) - f(100)}{0.01} \tag{7}$$

we would encounter first error when summing $100 + 0.01 = 100.01$. Due to `FD2` precision, it is trimmed to 100. Next, the function f is identity:

$$f(x)' = \frac{100 - 100}{0.01}$$
$$f(x)' = 0 \tag{8}$$

So, numerically we get the result of derivative to be 0, but we know that derivative of linear function is 1.

TODO: FD use 2x larger interval for better precision? maybe dont write

B. Symbolic method

A symbolic method was developed to tackle the problem with FD. It is precise. The method requires a set of known derivatives as well as set of rules (chain rule, product rule, …). For a given expression it uses the rules to expand it until only known derivatives are in the expression, then simplify the obtained expression and only then calculate the derivative.

$$f(x) = \frac{\sin(x)}{x}$$
using quotient rule:
$$\frac{g(x)}{h(x)} = \frac{g'(x)h(x) - g(x)h'(x)}{h(x)^2}$$
$$f'(x) = \frac{\sin(x)'x - \sin(x)x'}{x^2}$$
using known derivatives: (9)
$$\sin(x)' = \cos(x)$$
$$x' = 1$$
$$f'(x) = \frac{\cos(x)x - \sin(x)1}{x^2}$$
simplify:
$$f'(x) = \frac{x\cos(x) - \sin(x)}{x^2}$$

In Eq. (9) is demonstrated how symbolic method works. This method is used in programs such as Matlab, because it is precise. Note that this method has its drawbacks. The most important one is *expression swell*. The expression might become overly complex during differentiation because operations were not applied in advantageous order. While the result might be simple, the process might be not.

We will demonstrate expression swell on the function $f(x) = (x+1)^3$. Depending on how the symbolic solver program was written it might first expand the term as seen in Eq. (10). This approach in this particular case is better and leads to faster calculation and less computer memory usage.

$$f(x) = (x+1)^3 = x^3 + 3x^2 + 3x + 1$$
$$f'(x) = 3x^2 + 6x + 3 \tag{10}$$

If the solver begins by applying the product rule repeatedly as seen in Eq. (11), expression swell appears rather quickly. Depending on the solver it might simplify during the process so it might be little different than the example. Nevertheless, for more complex expressions it might not even be possible and the problem holds. The example should have demonstrated that even for very simple function, if differentiation done suboptimally, the computational cost is very high.

$$f(x) = (x+1)^3 = (x+1)(x+1)(x+1)$$
$$f'(x) = 1(x+1)(x+1) + (x+1)1(x+1) + (x+1)(x+1)1$$
$$f'(x) = (x+1)(x+1) + (x+1)(x+1) + (x+1)(x+1)$$
using: $(x+1)(x+1) = (x^2 + x + x + 1)$ (11)
$$f'(x) = (x^2 + x + x + 1) + (x^2 + x + x + 1) + (x^2 + x + x + 1)$$
$$f'(x) = x^2 + x + x + 1 + x^2 + x + x + 1 + x^2 + x + x + 1$$
$$f'(x) = 3x^2 + 6x + 3$$

C. Comparison

TODO: tabulka s porovnáním + vysvětlení proč

III. Automatic differentiation

TODO: že existují dvě varianty, začneme tím, co mají společného

A. Core principles

TODO: proč to funguje, teorie za tím

TODO: chain rule, výpočetní graf, potřeba mít známé funkce a jejich derivace

TODO: "jediný" rozdíl mezi FM/BM je pořadí derivací v chain rule - dá se vysvětlit i jako pořadí násobení jakobiánů

B. Forward mode: The tangent linear mode

TODO: důležitý je "přenášení" derivací spolu s hodnotou ve výpočtu

TODO: duální čísla ε^2 = 0 jako další matematický pohled na věc

TODO: vhodné pro málo vstupů, hodně výstupů - pro každý vstup třeba další průchod

C. Backward mode: The adjoint mode

TODO: potřeba dva průchody - první k vytvoření výpočetního grafu a zpětný k tomu, abychom viděli jak který vstup kontributuje k výsledné hodnotě (kterou si zvolíme) -> víc paměti. taky VJP (vector jacobian product) - a na tom ukázat, že je vhodné pro mnoho vstupů a málo výstupů (vs forward mode)

D. Computational complexity

TODO: časová a paměťová komplexita, porovnání s ostatními metodami

IV. Minimal implementation

TODO: jak dát do kódu, python - fundamentální rozdíl mezi FMode a BM

A. Forward mode

TODO: + vysvětlení pomocí duálních čísel, přetížení standardních algebraických operací tak, aby se přenášela i derivace

B. Backward mode

TODO: + vysvětlení jak se používá computational graf k zpětnému chodu

V. Using existing autodif libraries in python

TODO: předpokládám python + pytorch, tensorFlow

VI. Conclusion

TODO: jak cca funguje autodif, k čemu je dobrá (výhody oproti FD a symbolic), důležitost pro reálné aplikace, limitace

VII. Further reading

TODO: list mých zdrojů s lepším popisem, pro ty, které by zajímalo víc

[2], [3], [4]

References

[1] I. of Electrical and E. Engineers, "IEEE Standard for Floating-Point Arithmetic." [Online]. Available: https://standards.ieee.org/ieee/754/6210/

[2] Wikipedia contributors, "Automatic differentiation — Wikipedia, The Free Encyclopedia." [Online]. Available: https://en.wikipedia.org/w/index.php?title=Automatic_differentiation&oldid=1339245165

[3] Andrew Holmes, "What's Automatic Differentiation?." [Online]. Available: https://huggingface.co/blog/andmholm/what-is-automatic-differentiation

[4] Carnegie Mellon University, "Deep Learning Systems - Algorithms and Implementation." [Online]. Available: https://dlsyscourse.org/